Disjoint Sets, Union-Find and Kruskal's Algorithm

Set Representation, Path Compression, Union by Rank and Minimum Spanning Trees

Department of Computer Science and Engineering

Graphic Era (Deemed to be University)

What Is a Disjoint Set?

Definition

A disjoint-set data structure maintains a collection of non-overlapping sets.

Two sets A and B are disjoint if:

$$A \cap B = \varnothing$$

Example

$$S_1 = \{1, 2, 3\}$$

$$S_2 = \{4, 5\}$$

$$S_3 = \{6, 7, 8\}$$

No element belongs to more than one set.

Understanding Set Partitions

Let:

$$U = \{1, 2, 3, 4, 5, 6\}$$

A possible partition is:

$$S_1 = \{1, 2\}$$

$$S_2 = \{3, 4, 5\}$$

$$S_3 = \{6\}$$

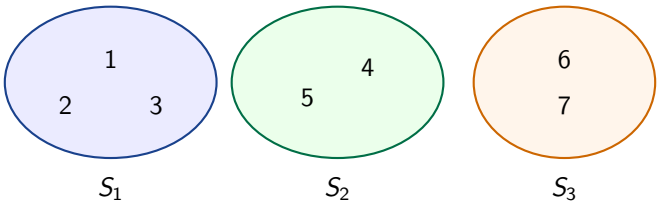
A valid partition satisfies:

$$S_i \cap S_j = \varnothing \quad \text{for } i \neq j$$

and:

$$\bigcup_i S_i = U$$

Disjoint Sets: Visual Representation



Observation

Every element belongs to exactly one set.

Why Do We Need Disjoint Sets?

Disjoint sets are useful when we need to:

- Maintain dynamically changing groups.
- Determine whether two elements belong to the same group.
- Merge two groups efficiently.
- Track connected components.
- Detect cycles in undirected graphs.
- Construct minimum spanning trees.

Application

Kruskal's algorithm uses Union-Find to avoid creating cycles.

Three Fundamental Operations

The disjoint-set data structure supports:

1. **MAKE-SET(x)**
   Create a new set containing only x.
2. **FIND(x)**
   Return the representative of the set containing x.
3. **UNION(x,y)**
   Merge the sets containing x and y.

MAKE-SET Operation

Definition

MAKE-SET creates a singleton set.

For example:

$$\textit{MAKE-SET}(1)$$

$$\textit{MAKE-SET}(2)$$

$$\textit{MAKE-SET}(3)$$

Results in:

$$\{1\}, \{2\}, \{3\}$$

Each element initially represents itself.

FIND Operation

Definition

FIND returns the representative or root of the set containing an element.

Suppose:

$$S_1 = \{1, 2, 3\}$$

with representative 1.
Then:

$$FIND(1) = 1$$

$$FIND(2) = 1$$

$$FIND(3) = 1$$

Important

The representative identifies the set; it need not be the smallest element.

UNION Operation

Definition

UNION merges two sets if their representatives differ.

Before:

$$S_1 = \{1, 2\}$$

$$S_2 = \{3, 4\}$$

After:

$$UNION(2, 4)$$

$$S = \{1, 2, 3, 4\}$$

Example: Sequence of Operations

Initially:

$$\{1\}, \{2\}, \{3\}, \{4\}, \{5\}$$

Perform:

$$UNION(1, 2)$$

$$UNION(3, 4)$$

$$UNION(2, 3)$$

Final partition:

$$\boxed{\{1, 2, 3, 4\}, \{5\}}$$

Question

Do 1 and 4 belong to the same set?

Yes, because:

Representing Disjoint Sets

Two common approaches are:

1. Array-based set labels.
2. Parent-pointer forests.

Parent-Pointer Representation

Each element stores a parent.
A root stores itself as parent.

$$parent[root] = root$$

Parent Array Example

Suppose:

$$parent = [0, 0, 0, 3, 3, 5]$$

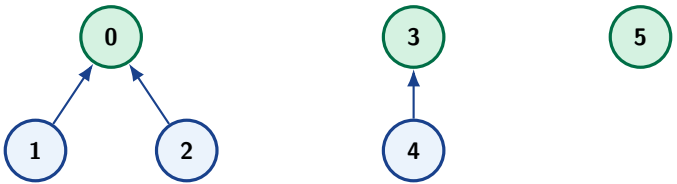
for elements 0 through 5.

Element	0	1	2	3	4	5
Parent	0	0	0	3	3	5

The sets are:

$$\{0, 1, 2\}, \quad \{3, 4\}, \quad \{5\}$$

Forest Representation



Observation

Each disjoint set is represented by a tree.
The collection of trees is a forest.

How FIND Works

To find the representative of x:

1. Examine $parent[x]$.
2. If $parent[x] = x$, return x.
3. Otherwise move to the parent.
4. Continue until reaching the root.

Example

$$parent[4] = 3$$

$$parent[3] = 3$$

Therefore:

$$FIND(4) = 3$$

C Code: Basic Union-Find

```c
#include <stdio.h>

#define MAX 100

int parent[MAX];

void makeSet(int n) {
    for (int i = 0; i < n; i++)
        parent[i] = i;
}

int findSet(int x) {
    if (parent[x] == x)
        return x;

    return findSet(parent[x]);
}

void unionSet(int x, int y) {
    int rootX = findSet(x);
    int rootY = findSet(y);

    if (rootX != rootY)
        parent[rootY] = rootX;
}
```

Problem with Basic Union-Find

Consider repeated unions that produce a long chain:

$$0 \leftarrow 1 \leftarrow 2 \leftarrow 3 \leftarrow 4$$

Finding the root of 4 requires following four parent links.

Problem

Unbalanced trees can make FIND operations slow.

In the worst case:

$$\boxed{O(V)}$$

What Is Path Compression?

Definition

Path compression makes vertices encountered during FIND point directly to the root.

Before:

$$4 \to 3 \to 2 \to 1 \to 0$$

After:

$$4 \to 0, \quad 3 \to 0, \quad 2 \to 0, \quad 1 \to 0$$

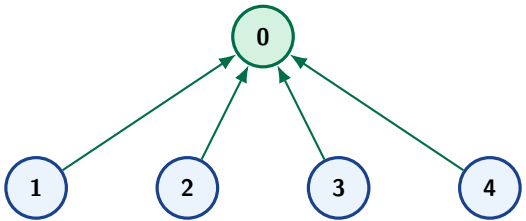
Future FIND operations become faster.

Path Compression: Before



$FIND(4) = 0$

Path Compression: After



Result

All visited vertices now point directly to the representative.

C Code: Path Compression

```c
int findSet(int x) {
    if (parent[x] != x) {
        parent[x] = findSet(parent[x]);
    }

    return parent[x];
}
```

Important Statement

```
parent[x] = findSet(parent[x]);
```

This assignment updates the parent during recursive unwinding.

Why Do We Need Union by Rank?

Without optimization, UNION may attach a large tree beneath a small tree.
This can increase tree height.

Union by Rank

Attach the root with smaller rank beneath the root with larger rank.

Rank is an upper bound on tree height when used with path compression.

Union by Rank: Rules

Let:

$$r_x = rank[rootX]$$

$$r_y = rank[rootY]$$

Then:

1. If $r_x < r_y$, attach X below Y.
2. If $r_x > r_y$, attach Y below X.
3. If ranks are equal, choose either root and increase its rank by 1.

Union by Rank: Example

Initially:

$$rank[A] = 0, \quad rank[B] = 0$$

After:

$$UNION(A, B)$$

choose A as root:

$$parent[B] = A$$

$$rank[A] = 1$$

Next Union

If a rank-1 tree merges with a rank-0 tree, the rank-1 root remains the root.

Union by Size

Another optimization is:

Union by Size

Attach the smaller set beneath the larger set.

Maintain:

$$size[root]$$

For example:

$$size[A] = 5$$

$$size[B] = 2$$

After merging:

$$parent[B] = A$$

$$size[A] = 7$$

Union by Rank vs Union by Size

Feature	Union by Rank	Union by Size
Criterion	Rank	Number of elements
Stored array	rank[]	size[]
Goal	Keep trees shallow	Keep trees shallow
Used with compression	Yes	Yes

Both approaches are effective.

Optimized Union-Find: Part 1

```c
#include <stdio.h>

#define MAX 100

int parent[MAX];
int rankArr[MAX];

void makeSet(int n) {
    for (int i = 0; i < n; i++) {
        parent[i] = i;
        rankArr[i] = 0;
    }
}

int findSet(int x) {
    if (parent[x] != x)
        parent[x] = findSet(parent[x]);

    return parent[x];
}
```

Optimized Union-Find: Part 2

```
void unionSet(int x, int y) {
    int rootX = findSet(x);
    int rootY = findSet(y);

    if (rootX == rootY)
        return;

    if (rankArr[rootX] < rankArr[rootY]) {
        parent[rootX] = rootY;
    }
    else if (rankArr[rootX] > rankArr[rootY]) {
        parent[rootY] = rootX;
    }
    else {
        parent[rootY] = rootX;
        rankArr[rootX]++;
    }
}
```

Optimized Union-Find: Main Program

```c
int main(void) {
    makeSet(6);

    unionSet(0, 1);
    unionSet(1, 2);
    unionSet(3, 4);

    printf("Find(2) = %d\n", findSet(2));
    printf("Find(4) = %d\n", findSet(4));

    if (findSet(0) == findSet(2))
        printf("0 and 2 are connected\n");

    if (findSet(0) != findSet(4))
        printf("0 and 4 are separate\n");

    unionSet(2, 4);

    if (findSet(0) == findSet(4))
        printf("0 and 4 are now connected\n");

    return 0;
}
```

Optimized Union-Find: Output

Expected Output

```
Find(2) = 0
Find(4) = 3
0 and 2 are connected
0 and 4 are separate
0 and 4 are now connected
```

Union-Find Complexity

With both path compression and union by rank or size:

$$\boxed{O(\alpha(V))}$$

amortized time per operation.
Here:

$$\alpha(V)$$

is the inverse Ackermann function.

Practical Meaning

It grows extremely slowly.
For realistic input sizes, operations are effectively constant-time on average.

What Is a Spanning Tree?

Definition

A spanning tree of a connected, undirected graph contains all vertices and has no cycles.

For V vertices, a spanning tree has:

$$\boxed{V - 1 \text{ edges}}$$

- Every vertex is connected.
- No cycle is present.
- Removing an edge disconnects the tree.

What Is a Minimum Spanning Tree?

Definition

A minimum spanning tree (MST) is a spanning tree with minimum possible total edge weight.

If selected edges are:

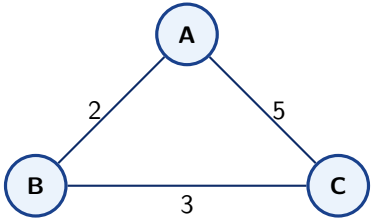
$$e_1, e_2, \ldots, e_{V-1}$$

then MST minimizes:

$$\boxed{W(T) = \sum_{e \in T} w(e)}$$

subject to connectivity and acyclicity.

MST Example



Possible spanning-tree weights:

$$2 + 3 = 5$$

$$2 + 5 = 7$$

$$3 + 5 = 8$$

$$\boxed{MST\ Weight = 5}$$

Applications of MST

Minimum spanning trees are useful in:

1. Computer network design.
2. Electrical distribution.
3. Road and railway planning.
4. Communication infrastructure.
5. Cable layout optimization.
6. Clustering applications.

Two Popular MST Algorithms

Kruskal's Algorithm

- Sort edges by weight.
- Select safe edges.
- Reject cycles.
- Uses Union-Find.

Prim's Algorithm

- Start at a vertex.
- Grow one tree.
- Select minimum crossing edge.
- Uses a priority queue or matrix scanning.

Prim's algorithm will be discussed in the next presentation.

What Is Kruskal's Algorithm?

Definition

Kruskal's algorithm is a greedy algorithm that constructs an MST by repeatedly selecting the minimum-weight edge that connects two different components.

- Sort edges by increasing weight.
- Initially, every vertex is separate.
- Accept edges that do not form cycles.
- Stop after selecting $V - 1$ edges.

Why Is Kruskal's Greedy?

At every step, Kruskal's algorithm chooses the lightest available edge that joins two different components.

Greedy Choice

Select the lowest-weight safe edge available.

Important

A low-weight edge is rejected if its endpoints are already connected.

Kruskal's Algorithm: Steps

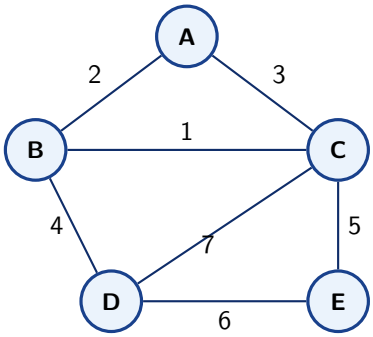
1. Initialize each vertex as a separate set.
2. Sort all edges in ascending order of weight.
3. Examine edges one by one.
4. For edge (u, v), compute FIND(u) and FIND(v).
5. If roots differ, accept the edge.
6. Merge the two sets using UNION.
7. Otherwise reject the edge.
8. Stop when $V - 1$ edges are selected.

Kruskal's Pseudocode

Algorithm: Kruskal(G)

```
MST = empty set
For each vertex v:
  MAKE-SET(v)
Sort edges by increasing weight
For each edge (u,v,w):
  If FIND(u) != FIND(v):
    Add (u,v,w) to MST
    UNION(u,v)
  If MST has V-1 edges:
    Stop
```

Worked Example: Weighted Graph



Task

Find an MST using Kruskal's algorithm.

Step 1: Sort Edges

Edge	Weight
B–C	1
A–B	2
A–C	3
B–D	4
C–E	5
D–E	6
C–D	7

Initial State

$$\{A\}, \{B\}, \{C\}, \{D\}, \{E\}$$

Step 2: Select B–C

Edge:

$$(B, C), \quad w = 1$$

B and C are in different sets.
Accept the edge.

$$MST = \{(B, C)\}$$

$$W = 1$$

Updated sets:

$$\{A\}, \{B, C\}, \{D\}, \{E\}$$

Step 3: Select A–B

Edge:

$$(A, B), \quad w = 2$$

A and B belong to different sets.
Accept the edge.

$$MST = \{(B, C), (A, B)\}$$

$$W = 1 + 2 = 3$$

Updated sets:

$$\{A, B, C\}, \{D\}, \{E\}$$

Step 4: Reject A–C

Edge:

$$(A, C), \quad w = 3$$

A and C already belong to the same connected component.

$$FIND(A) = FIND(C)$$

Decision

Reject the edge.
Adding it would form a cycle.

$$W = 3$$

Step 5: Select B–D

Edge:

$$(B, D), \quad w = 4$$

B and D belong to different sets.
Accept the edge.

$$MST = \{(B, C), (A, B), (B, D)\}$$

$$W = 1 + 2 + 4 = 7$$

Updated sets:

$$\{A, B, C, D\}, \{E\}$$

Step 6: Select C–E

Edge:

$$(C, E), \quad w = 5$$

C and E belong to different sets.
Accept the edge.

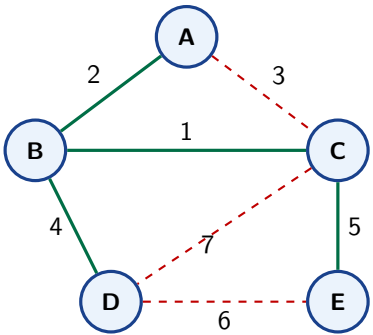
$$MST = \{(B, C), (A, B), (B, D), (C, E)\}$$

$$W = 1 + 2 + 4 + 5$$

$$\boxed{W = 12}$$

Four edges have been selected for five vertices.
Stop.

Final Minimum Spanning Tree



$$\boxed{MST\ \ Weight = 12}$$

Green edges form the MST.

Complete Kruskal Dry Run

Step	Edge	Weight	Decision	Total
1	B–C	1	Accept	1
2	A–B	2	Accept	3
3	A–C	3	Reject	3
4	B–D	4	Accept	7
5	C–E	5	Accept	12

Result

$$\boxed{\textit{MST Weight} = 12}$$

$$\boxed{\textit{Selected Edges} = 4}$$

C Implementation Strategy

The C program requires:

1. An edge structure containing source, destination and weight.
2. Sorting edges by weight.
3. A Union-Find data structure.
4. A loop to select safe edges.
5. Total MST weight calculation.

Implementation

We will use:

`qsort()`

and optimized Union-Find.

Kruskal in C: Part 1

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXV 100
#define MAXE 1000

typedef struct {
    int u, v, weight;
} Edge;

int parent[MAXV];
int rankArr[MAXV];

void makeSet(int n) {
    for (int i = 0; i < n; i++) {
        parent[i] = i;
        rankArr[i] = 0;
    }
}

int findSet(int x) {
    if (parent[x] != x)
        parent[x] = findSet(parent[x]);

    return parent[x];
}
```

Kruskal in C: Part 2

```c
void unionSet(int x, int y) {
    int rx = findSet(x);
    int ry = findSet(y);

    if (rx == ry)
        return;

    if (rankArr[rx] < rankArr[ry]) {
        parent[rx] = ry;
    }
    else if (rankArr[rx] > rankArr[ry]) {
        parent[ry] = rx;
    }
    else {
        parent[ry] = rx;
        rankArr[rx]++;
    }
}

int compareEdges(const void *a, const void *b) {
    const Edge *x = (const Edge *)a;
    const Edge *y = (const Edge *)b;

    return (x->weight > y->weight)
         - (x->weight < y->weight);
}
```

Kruskal in C: Part 3

```c
void kruskal(Edge edges[], int V, int E) {
    qsort(edges, E, sizeof(Edge), compareEdges);

    makeSet(V);

    int selected = 0;
    long long total = 0;

    printf("Selected MST Edges:\n");

    for (int i = 0; i < E && selected < V - 1; i++) {
        int u = edges[i].u;
        int v = edges[i].v;

        if (findSet(u) != findSet(v)) {
            unionSet(u, v);

            printf("%c - %c : %d\n",
                   'A' + u, 'A' + v,
                   edges[i].weight);

            total += edges[i].weight;
            selected++;
        }
    }

    if (selected != V - 1) {
        printf("Graph is disconnected.\n");
        printf("No spanning tree exists.\n");
    } else {
        printf("Total MST Weight: %lld\n", total);
    }
}
```

Kruskal in C: Main Program

```c
int main(void) {
    int V = 5;

    Edge edges[] = {
        {1, 2, 1},
        {0, 1, 2},
        {0, 2, 3},
        {1, 3, 4},
        {2, 4, 5},
        {3, 4, 6},
        {2, 3, 7}
    };

    int E = sizeof(edges) / sizeof(edges[0]);

    kruskal(edges, V, E);

    return 0;
}
```

Kruskal C Program: Output

Expected Output

```
Selected MST Edges:
B - C : 1
A - B : 2
B - D : 4
C - E : 5
Total MST Weight:  12
```

Kruskal's Time Complexity

Main operations:

1. Sort all edges.
2. Perform FIND operations.
3. Perform UNION operations.

Sorting requires:

$$O(E \log E)$$

Union-Find operations require:

$$O(E\alpha(V))$$

Overall:

$$\boxed{O(E \log E)}$$

for a graph with E edges.

Kruskal's Space Complexity

Kruskal's algorithm stores:

- Edge list.
- Parent array.
- Rank array.
- Selected MST edges.

$$\boxed{O(V + E)}$$

total storage for the standard edge-list implementation.
Auxiliary Union-Find storage:

$$\boxed{O(V)}$$

When Is Kruskal's Algorithm Useful?

Kruskal's algorithm is especially convenient when:

- The graph is stored as an edge list.
- Edges can be sorted efficiently.
- The graph is relatively sparse.
- Union-Find is available.
- We need an MST or minimum spanning forest.

Kruskal's Algorithm: Important Properties

- Works on weighted undirected graphs.
- Produces an MST if the graph is connected.
- Produces a minimum spanning forest if the graph is disconnected.
- Can handle negative edge weights.
- May have multiple valid MSTs when edge weights are repeated.
- Uses greedy selection and cycle detection.

Exercise 1: Disjoint Sets

Question

Initially:

$$\{1\}, \{2\}, \{3\}, \{4\}, \{5\}$$

Perform:

$$UNION(1, 2)$$

$$UNION(3, 4)$$

$$UNION(2, 3)$$

How many sets remain?

Answer

$$\{1, 2, 3, 4\}, \{5\}$$

Exercise 2: FIND Operation

Question

Given:

$$parent = [0, 0, 1, 2, 4]$$

Find the representative of element 3.

Solution

$$3 \to 2 \to 1 \to 0$$

$$\boxed{FIND(3) = 0}$$

Exercise 3: Kruskal

Question

A weighted triangle contains:

$$w(A, B) = 4$$

$$w(B, C) = 2$$

$$w(A, C) = 5$$

Find its MST weight.

Solution

Choose B–C and A–B.

$$\boxed{MST\ Weight = 2 + 4 = 6}$$

Exercise 4: Spanning Tree

Question

A connected graph contains 12 vertices.
How many edges will its MST contain?

Solution

$$V - 1 = 12 - 1$$

$$\boxed{11}$$

MCQ 1

Which operation returns the representative of a set?

1. UNION
2. FIND
3. PUSH
4. ENQUEUE

Correct Answer

Option 2: FIND.

MCQ 2

Path compression is used to:

1. Increase tree height
2. Reduce future FIND costs
3. Sort graph edges
4. Find shortest paths

Correct Answer

Option 2.

MCQ 3

Union by rank attempts to:

1. Keep disjoint-set trees shallow
2. Increase the number of components
3. Sort vertices alphabetically
4. Minimize edge weights directly

Correct Answer

Option 1.

MCQ 4

Kruskal's algorithm uses:

1. BFS
2. DFS only
3. Union-Find
4. Binary search

Correct Answer

Option 3: Union-Find.

MCQ 5

A spanning tree with V vertices contains:

1. V edges
2. $V + 1$ edges
3. $V - 1$ edges
4. V^2 edges

Correct Answer

Option 3: $V - 1$.

MCQ 6

Kruskal rejects an edge when:

1. Its weight is negative
2. Its endpoints are already connected
3. It is the smallest edge
4. It connects different components

Correct Answer

Option 2.

Key Concepts

Concept	Main Purpose
MAKE-SET	Create singleton set
FIND	Identify representative
UNION	Merge two sets
Path compression	Shorten parent paths
Union by rank	Control tree height
Union by size	Attach smaller set
Kruskal	Construct MST

Complexity Summary

Operation / Algorithm	Complexity
MAKE-SET	$O(1)$ per element
Optimized FIND	$O(\alpha(V))$ amortized
Optimized UNION	$O(\alpha(V))$ amortized
Kruskal sorting	$O(E \log E)$
Kruskal overall	$O(E \log E)$

Presentation Summary

- Disjoint sets maintain non-overlapping groups.
- MAKE-SET, FIND and UNION are fundamental operations.
- Parent arrays represent disjoint-set forests.
- Path compression improves FIND efficiency.
- Union by rank or size keeps trees shallow.
- Kruskal constructs a minimum spanning tree.
- Edges are processed in increasing weight order.
- Union-Find prevents cycle formation.
- Kruskal runs in $O(E \log E)$ time.

Questions for Revision

1. Define a disjoint set.
2. Explain MAKE-SET, FIND and UNION.
3. Describe parent-array representation.
4. Explain path compression with an example.
5. Explain union by rank.
6. Compare union by rank and union by size.
7. Define a minimum spanning tree.
8. Explain Kruskal's algorithm.
9. Perform a Kruskal dry run.
10. Write the C implementation of Kruskal.

References

1. Reema Thareja, *Data Structures Using C*.
2. Thomas H. Cormen et al., *Introduction to Algorithms*.
3. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C*.

Minimum Spanning Trees, Prim's and Dijkstra's Algorithms

- Prim's greedy algorithm.
- Step-by-step MST construction.
- Prim's C implementation.
- Prim's vs Kruskal's algorithm.
- Dijkstra's shortest-path algorithm.
- Distance and parent arrays.
- Detailed shortest-path dry runs.
- C implementation and complexity.

Student Feedback

Share Your Feedback



Thank you!